

第13讲：函数、斜率与导数

为 Gradient 与 Backpropagation 补上的最小微积分

- 只学深度学习真正需要的微积分直觉，不先学完整高等数学。
- 从“参数改一点，输出会改多少？”出发理解导数。
- 通过手算、数值差分和图形实验理解局部斜率。
- 主动连接 Weight、Activation、Gradient、Backpropagation 和 LLM 训练。

本讲最低目标

学完以后，你不需要会证明极限，但必须能够用自己的话解释：函数是什么、斜率是什么、导数为什么是“局部敏感度”，以及为什么深度学习需要它。

1. 为什么现在必须补一点微积分？

【原理】

到第12讲为止，我们已经能完成完整 Forward：给定 Weight 和 Bias，输入经过 Linear、Activation、MLP 得到输出。但训练还有一个关键问题：模型怎么知道每个参数应该往哪个方向改、改多少？要回答这个问题，就需要“变化率”。导数正是测量局部变化率的工具。

先把问题缩小到最简单的情况。假设一个神经元：

$$y = w * x + b$$

如果当前 $x=2.5$ 、 $b=0.4$ 、 $w=1.2$ ，那么 $y=3.4$ 。现在把 w 从 1.20 改成 1.21，输出会变成多少？变化快不快？这就是导数要回答的事情。

【通俗解释】

把函数想成一台机器：输入一个数字，输出另一个数字。导数不是研究整台机器“长什么样”，而是在某个具体工作点附近问：我把输入旋钮拧一点点，输出指针大约会动多少？

2. 函数 Function：先理解“输入 -> 规则 -> 输出”

函数可以先不用数学化地理解成一条确定规则。给定输入 x ，按照某个规则得到输出 y 。写作：

$$y = f(x)$$

例如 $f(x)=3x+2$ 。输入 $x=4$ ：

$$f(4) = 3 * 4 + 2 = 14$$

在深度学习里，几乎每个模块都可以看成函数：Linear 是函数，ReLU 是函数，一个 MLP 是函数，一个 Transformer Block 也是函数，整个 LLM 也可以看成一个极其复杂的函数。

与前课连接

你在第11-12讲看到的 Forward，本质上就是把多个函数串起来： $x \rightarrow \text{Linear1} \rightarrow \text{ReLU} \rightarrow \text{Linear2} \rightarrow y$ 。后面 Backpropagation 要研究的，就是这些函数之间“变化率”怎样层层传递。

3. 斜率 Slope：先从直线开始

直线 $y=3x+2$ 很容易理解：x 每增加 1，y 就增加 3。所以它的斜率是 3。两点之间的平均斜率写成：

$$\text{slope} = (y_2 - y_1) / (x_2 - x_1) = \Delta y / \Delta x$$

具体数字例子

取 $x_1=1$ ，则 $y_1=5$ ；取 $x_2=4$ ，则 $y_2=14$ 。

$$\text{slope} = (14 - 5) / (4 - 1) = 9/3 = 3$$

对直线来说，不管你在哪两个点之间算，斜率都一样。

【必要公式】

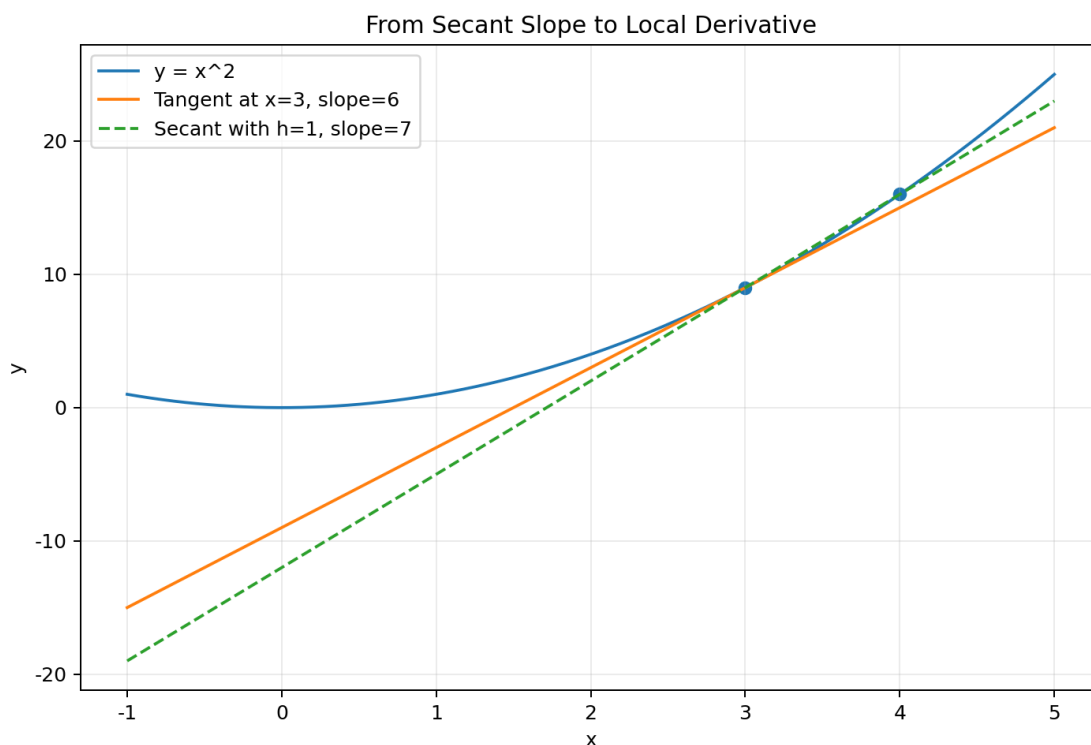
斜率的核心不是“背一个分数”，而是理解：输入变化了多少，输出随之变化了多少。
分子是输出变化，分母是输入变化。

4. 曲线的问题：斜率不再是常数

现在看曲线 $y=x^2$ 。x 从 0 到 1，y 从 0 到 1；x 从 2 到 3，y 从 4 到 9。显然不同位置“陡峭程度”不同。所以我们不再满足于问“整段平均斜率是多少”，而要问：在 $x=3$ 这个点附近，曲线此刻到底有多陡？

$$\text{average slope} = [f(x+h) - f(x)] / h$$

这里 h 是一个小变化。如果 h 越来越小，两点越来越靠近，割线就越来越接近该点的切线。



图解结论

在 $y=x^2$ 、 $x=3$ 处， $h=1$ 时割线斜率为 7； h 越小，斜率越接近 6。这个“无限靠近时的局部斜率”就是导数的直觉。

5. 导数 Derivative：局部变化率 / 局部敏感度

【原理】

导数描述：在当前点附近，输入发生很小变化时，输出会以多快的速度变化。

常见写法有：

$$f'(x) \square dy/dx$$

对于 $y=x^2$ ，它的导数是：

$$f'(x) = 2x$$

所以在 $x=3$ ：

$$f'(3) = 2*3 = 6$$

这不意味着“ x 增加 1, y 一定增加 6”。导数描述的是局部关系。当变化足够小时：

$$\Delta y \text{ approximately } f'(x) * \Delta x$$

一个极小变化的例子

$x=3$ 时，导数为 6。如果 x 增加 0.01，那么导数会预测：

$$\Delta y \text{ approximately } 6 * 0.01 = 0.06$$

真实变化是 $3.01^2 - 3^2 = 0.0601$ ，非常接近 0.06。变化越小，这种局部线性近似通常越准确。

6. 导数为什么与 Weight 直接相关？

回到最简单的神经元：

$$y = w*x + b$$

现在不动 x 和 b ，只改变 w 。我们问： w 改一点， y 会改多少？

因为：

$$\Delta y = x * \Delta w$$

所以 y 对 w 的变化率是：

$$dy/dw = x$$

具体数字例子

若 $x=2.5$ ，则 $dy/dw=2.5$ 。把 w 增加 0.01：

$$\Delta y \text{ approximately } 2.5 * 0.01 = 0.025$$

科研直觉

深度学习里 Weight 可能有数百万甚至数十亿个。训练的本质不是随便试参数，而是要知道：每个参数向上或向下动一点，会怎样影响最终 Loss。
后面“Gradient”就是把大量这种局部敏感度组织起来。

7. 导数的正负和大小分别告诉我们什么？

导数	直觉	如果输入略微增加
> 0	当前是上坡	输出倾向增加
< 0	当前是下坡	输出倾向减小
$= 0$	局部平坦	输出一阶变化很小
绝对值大	很敏感/很陡	小输入变化可带来较大输出变化
绝对值小	不太敏感/较平	小输入变化影响较弱

注意：导数为0并不自动等于“找到了最优解”。以后学 Loss Landscape 时会看到局部极小值、局部极大值、鞍点和平坦区域。

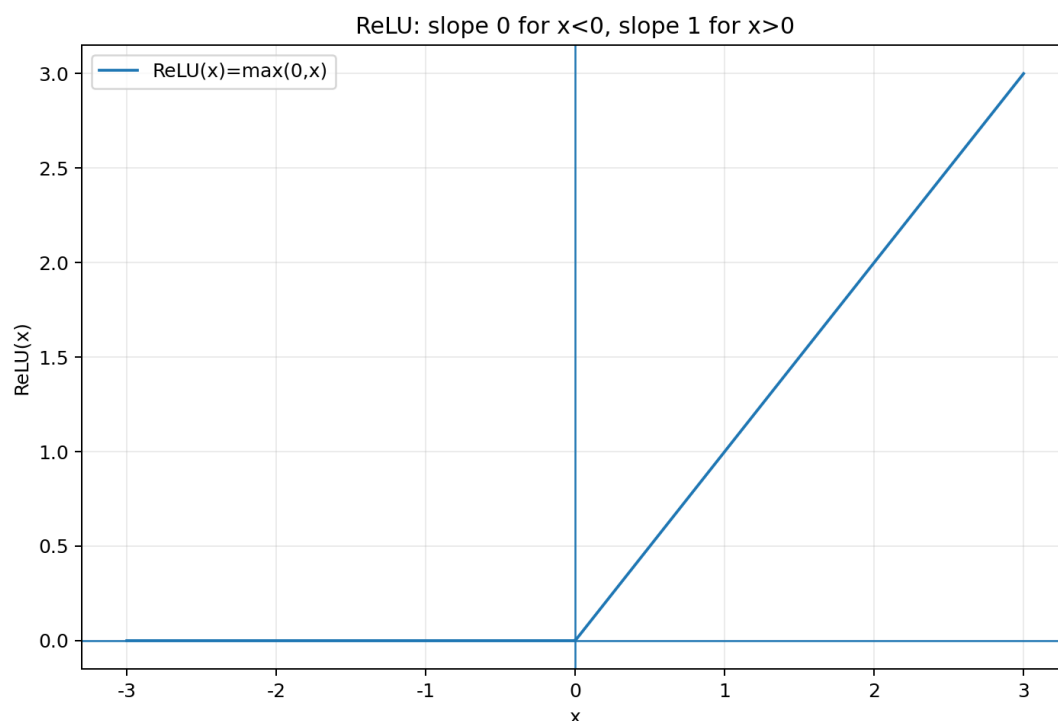
8. ReLU 的导数：为什么 Activation 也参与 Backward？

第9讲学过：

$$\text{ReLU}(x) = \max(0, x)$$

在 $x < 0$ 时，它是一条水平线，所以斜率为 0；在 $x > 0$ 时，它就是 $y=x$ ，所以斜率为 1。

$$\text{ReLU}'(x) = 0 \quad (x < 0), \quad 1 \quad (x > 0)$$



$x=0$ 的特殊情况

ReLU 在 0 处有一个“折角”：左侧斜率是 0，右侧斜率是 1，所以严格数学意义上导数不存在。深度学习框架会采用约定值来完成计算。当前只需知道：现实神经网络里会遇到这种分段可导情况。

9. 数值导数：不用会推公式也能估计导数

我们可以直接让计算机把 x 向左右各挪一点：

$$f'(x) \text{ approximately } [f(x+h) - f(x-h)] / (2h)$$

这叫中心差分。它不是训练大型网络的正式方法，但非常适合现在建立直觉，也常被用来做 Gradient Check。

Python / NumPy 最小代码

```
def numerical_derivative(f, x, h=1e-5):  
    # x 0.0000000000  
    right = f(x + h)  
    # x 0.0000000000  
    left = f(x - h)  
    # 0.00000000000000  
    return (right - left) / (2 * h)  
  
def f(x):  
    return x ** 2  
  
print(numerical_derivative(f, 3.0)) # 6
```

常见误区

h 不是越小越无限好。计算机使用有限精度浮点数， h 极端小时会出现数值误差。当前只需理解“足够小的有限差分可以近似导数”。

10. 从 Derivative 到 Gradient：今天先看地图，不提前跳课

本讲只处理“一个输入变量 $\rightarrow$ 一个输出”的导数。神经网络却有很多 Weight。比如 Loss 同时依赖 w_1 、 w_2 、 w_3 ……

下一阶段我们会引入 Partial Derivative 与 Gradient：

$$dL/dw_1, dL/dw_2, dL/dw_3, \dots$$

可以先把它们理解为：分别问“只把这个参数轻轻动一点，Loss 会怎么变？”

不要现在硬背

Partial Derivative、Gradient、Chain Rule 会在后面单独逐步展开。本讲只要求把“导数 = 局部敏感度”建立牢固。

11. 与 LLM / Transformer / Agent Systems 的关系

11.1 LLM Pretraining

大模型 Forward 后得到 Next-token Prediction Loss。训练必须知道每个参数怎样影响这个 Loss，于是需要梯度，而梯度的最小构件就是导数/偏导数。

11.2 Activation

GELU、SiLU、ReLU 不只在 Forward 时改变数值；在 Backward 时，它们自己的局部导数也会影响梯度能不能继续向前面的层传播。

11.3 为什么会有 Vanishing / Exploding Gradient

以后你会看到，很多层的局部变化率在 Chain Rule 中相乘。如果大量因子小于 1，梯度可能越来越小；如果很多因子很大，梯度可能爆炸。本讲就是这些问题的数学起点。

11.4 Agent Systems

Agent Systems 研究通常不要求你自己推导所有神经网络公式，但当论文讨论 fine-tuning、LoRA、optimizer、training instability 或 calibration 时，你至少必须知道参数更新背后的局部敏感度概念。

12. 常见误区

误区 1：导数就是 “ y/x ”

不是。导数是变化量之比 $\Delta y / \Delta x$ 在局部极限下的概念；它取决于当前点。

误区 2：导数大就一定 “重要”

不一定。它只说明在当前点、当前尺度下局部敏感。因果重要性、全局重要性是另外的问题。

误区 3：导数为 0 就是最优点

不一定。可能是极小值、极大值、鞍点或平坦区。

误区 4：数值导数就是 Backpropagation

不是。数值差分需要反复重新算 Forward，成本极高；Backpropagation 会系统利用计算图和 Chain Rule 高效求梯度。

13. 本讲实验：把导数“看见”

本讲实验不是为了练复杂代码，而是验证五个问题：

1) 直线的局部斜率是否处处相同；2) x^2 的斜率是否随位置改变；3) h 变小时割线是否逼近切线；4) Weight 的导数是否能预测输出变化；5) ReLU 左右两侧斜率是否不同。

实验方法

先在纸上预测，再运行代码，再把输出与理论值对照。不要只看“代码运行成功”。

实验中最重要的一行代码

```
return (f(x + h) - f(x - h)) / (2 * h)
```

这行代码把“导数 = 局部变化率”的概念转成了可以运行的实验。

14. 本讲最低验收标准

必须真正掌握

① 能说清函数是“输入-规则-输出”；② 能手算两点斜率；③ 能解释导数是当前点附近的局部变化率；④ 看懂 $f'(x)$ 与 dy/dx ；⑤ 能解释导数的正负和大小；⑥ 能说明为什么训练 Weight 需要导数；⑦ 能用数值差分估计一个简单函数的导数。

一句话总图

parameter changes a little -> output/Loss changes -> derivative
measures sensitivity -> gradient organizes many sensitivities ->
backprop computes them efficiently

下一讲预告

第14讲将正式进入 Loss

Function：模型到底怎样把“预测错了多少”压缩成一个可优化的数字。你会看到导数为什么必须和 Loss 连接起来。